

SQLexer — Analizador Léxico

Proyecto de Programación de Sistemas de Base 1

Campo	Valor
Materia	Programación de Sistemas de Base 1
Institución	Universidad Autónoma de Tamaulipas
Semestre	2026-1
Profesor	Muñoz Quintero Dante Adolfo
Integrantes	Macías Tijerina Guillermo Antonio Pecero Acevedo Héctor Humberto Ponce Medina Ángel Jiménez Blanco José Antonio
Repositorio	https://github.com/GuillermoMacias28/Analizador-lexico-SQLexer
Fecha	Mayo 2026

El presente documento constituye la Demostración Funcional del proyecto de semestre para la materia de Programación de Sistemas de Base 1. Incluye la especificación del lenguaje diseñado, el código fuente comentado del analizador léxico y capturas de ejecución con casos válidos y con errores.

Tabla de contenido

Sección 1 — Introducción	3
Sección 2 — Objetivos	4
Objetivo general	4
Objetivos específicos	4
Sección 3 — Especificación del Lenguaje.....	5
3.1 Nombre y propósito	5
3.2 Ejemplos ilustrativos de código	5
3.3 Catálogo de tokens	6
Sección 4 — Implementación	9
4.1 Definición de tokens y palabras reservadas — SQL.g4.....	9
4.2 Reglas léxicas — SQL.g4 (archivos generados por ANTLR)	10
4.3 Lógica del analizador léxico — SQLLexer.py	11
4.4 Manejo y reporte de errores — main.py	11
Sección 5 — Demostración Funcional.....	13
5.1 Ejecución con consulta válida.....	13
5.2 Ejecución con errores léxicos	15
Sección 6 — Conclusiones.....	17
Sección 7 — Referencias	18

Sección 1 — Introducción

El análisis léxico es la primera fase de un compilador o intérprete: recibe una secuencia de caracteres y produce una secuencia de tokens que representan las unidades mínimas con significado del lenguaje.

Comprender esta etapa de forma práctica es fundamental en la formación de un ingeniero en sistemas computacionales que desee construir herramientas de procesamiento de lenguajes.

El presente proyecto diseña e implementa un analizador léxico para SQLexer, un lenguaje basado en SQL (Structured Query Language), el estándar internacional para consulta y manipulación de bases de datos relacionales. El nombre SQLexer combina "SQL" con "lexer", reflejando directamente el propósito del proyecto: analizar léxicamente instrucciones SQL y transformarlas en una lista clasificada de tokens.

El lexer fue implementado en Python 3 utilizando ANTLR4 (ANother Tool for Language Recognition), una herramienta de generación automática de analizadores. Con ANTLR4, la gramática del lenguaje se define en un archivo .g4 y la herramienta genera automáticamente el lexer, el parser, el listener y el visitor en Python. Se utilizó el runtime antlr4-python3-runtime para la ejecución.

Este documento se organiza de la siguiente manera: la Sección 2 presenta los objetivos del proyecto; la Sección 3 describe la especificación del lenguaje SQLexer; la Sección 4 contiene el código fuente comentado; la Sección 5 muestra capturas de ejecución; finalmente las Secciones 6 y 7 presentan conclusiones y referencias.

Sección 2 — Objetivos

Objetivo general

Diseñar un lenguaje basado en SQL e implementar un analizador léxico funcional utilizando ANTLR4, que transforme consultas SQL en listas de tokens clasificados por categoría semántica, con manejo preciso de errores léxicos.

Objetivos específicos

1. Definir la especificación del lenguaje SQLexer mediante ejemplos ilustrativos de consultas SQL que cubran sus construcciones principales (DML, DDL, TCL, DCL, funciones, tipos de dato).
2. Establecer un catálogo completo de tokens organizado por categoría: palabras reservadas, identificadores, números, operadores, delimitadores, cadenas de texto y funciones.
3. Implementar el analizador léxico utilizando ANTLR4, aprovechando la generación automática del lexer a partir de la gramática .g4.
4. Implementar la detección y reporte de errores léxicos, indicando número de línea, columna y descripción del problema para cada error encontrado.
5. Probar el lexer con consultas SQL válidas y con entradas que contengan errores léxicos, verificando la correcta identificación y reporte de los mismos.

Sección 3 — Especificación del Lenguaje

3.1 Nombre y propósito

SQLexer es un analizador léxico para el lenguaje SQL (Structured Query Language), el estándar internacional para consulta y manipulación de bases de datos relacionales. El nombre combina "SQL" y "lexer" para reflejar directamente su función. La extensión de los archivos de entrada es .sql.

SQLexer reconoce instrucciones de cuatro categorías principales de SQL: DML (manipulación de datos: SELECT, INSERT, UPDATE, DELETE), DDL (definición de estructura: CREATE, ALTER, DROP), TCL (control de transacciones: BEGIN, COMMIT, ROLLBACK) y DCL (control de permisos: GRANT, REVOKE).

3.2 Ejemplos ilustrativos de código

Ejemplo 1 — Consulta con JOIN y funciones de ventana:

```
WITH EstadisticasDept AS (  
    SELECT  
        e.dept_id,  
        COUNT(e.id) AS total_empleados,  
        AVG(e.salario) OVER (PARTITION BY e.dept_id) AS salario_promedio,  
        CASE  
            WHEN e.salario > 100000 THEN 'Alto'  
            ELSE 'Normal'  
        END AS clasificacion  
    FROM empleados AS e  
    WHERE e.activo = TRUE  
    GROUP BY e.dept_id, e.salario  
)
```

Ejemplo 2 — Consulta con errores léxicos:

```
DECLARE @mi_variable INT;

SELECT id, nombre, salario$
FROM empleados#temporales
WHERE activo = TRUE;

WITH Ventas AS (
    SELECT
        vendedor_id,
        total ~ descuento AS total_real
    FROM ventas_diarias
)
SELECT * FROM Ventas;

GRANT SELECT ON ?tabla_desconocida TO usuario;
```

3.3 Catálogo de tokens

Categoría	Ejemplos	Descripción
DML	SELECT, INSERT, UPDATE, DELETE	Operaciones de manipulación de datos
DDL	CREATE, ALTER, DROP, TABLE	Instrucciones de definición de estructura
TCL	BEGIN, COMMIT, ROLLBACK	Control de transacciones
DCL	GRANT, REVOKE, DENY, TO	Control de permisos

CLAUSULA	FROM, WHERE, JOIN, GROUP BY, ORDER BY, HAVING, WITH, AS	Cláusulas modificadoras de consulta
CONDICIONAL	CASE, WHEN, THEN, ELSE, END, BETWEEN, LIKE, IN, IS, NOT, EXISTS	Lógica condicional
FUNCION	COUNT, SUM, AVG, MAX, MIN, UPPER, LOWER, CAST, NOW, COALESCE	Funciones SQL integradas
VENTANA	OVER, PARTITION	Funciones de ventana
TIPO_DATO	INT, VARCHAR, DECIMAL, DATE, BOOLEAN, TEXT, FLOAT, TIMESTAMP	Tipos de dato
OP_RELACIONAL	=, <>, !=, <, >, <=, >=	Operadores de comparación
OP_ARITMETICO	+, -, *, /	Operadores aritméticos
OP_LOGICO	AND, OR, NOT	Operadores lógicos
DELIMITADOR	;, ,, (,), ., { }	Separadores y agrupadores
IDENTIFICADOR	empleados, dept_id, mi_tabla	Nombres de tablas, columnas, alias
NUMERO_ENTERO	0, 42, 100000	Secuencia de dígitos sin punto decimal
NUMERO_DECIMAL	3.14, 0.5, 1.0	Dígitos, punto, dígitos obligatorios

CADENA	'Alto', 'Normal'	Cadena entre comillas simples
ERROR_LEXICO	@, #, ~, \$, ?	Símbolo no reconocido por la gramática

Identificadores válidos: comienzan con letra (a-z, A-Z) o guión bajo (_), seguidos de cero o más letras, dígitos o guiones bajos. Ejemplo: mi_tabla, empleados2, _aux.

Identificadores inválidos (generan error): contienen caracteres especiales no permitidos como @, #, ~, \$ o ?.

Sección 4 — Implementación

A continuación se presenta el código fuente de los módulos del analizador léxico. En SQLexer, ANTLR4 genera automáticamente el lexer, el parser, el listener y el visitor a partir de un único archivo de gramática SQL.g4.

4.1 Definición de tokens y palabras reservadas — SQL.g4

El archivo SQL.g4 es la gramática del lenguaje. Contiene todas las reglas que definen los tokens de SQL. Las reglas del lexer van en MAYÚSCULAS. La técnica más importante es el uso de corchetes por letra para lograr insensibilidad a mayúsculas sin flags especiales: SELECT se define como [Ss][Ee][Ll][Ee][Cc][Tt], lo que acepta select, SELECT o SeLeCt como el mismo token. El operador | permite que un token reconozca sinónimos: INT e INTEGER producen el mismo tipo de token.

```
grammar SQL;

// Reglas de parser (requeridas por ANTLR)
programa : sentencia* EOF ;
sentencia : . ;

// — DML —
SELECT : [Ss][Ee][Ll][Ee][Cc][Tt] ;
INSERT : [Ii][Nn][Ss][Ee][Rr][Tt] ;
UPDATE : [Uu][Pp][Dd][Aa][Tt][Ee] ;
DELETE : [Dd][Ee][Ll][Ee][Tt][Ee] ;

// — CLÁUSULAS —
FROM : [Ff][Rr][Oo][Mm] ;
WHERE : [Ww][Hh][Ee][Rr][Ee] ;
WITH : [Ww][Ii][Tt][Hh] ;
AS : [Aa][Ss] ;
JOIN : [Jj][Oo][Ii][Nn] ;
ON : [Oo][Nn] ;
GROUP : [Gg][Rr][Oo][Uu][Pp] ;
BY : [Bb][Yy] ;
ORDER : [Oo][Rr][Dd][Ee][Rr] ;
HAVING : [Hh][Aa][Vv][Ii][Nn][Gg] ;

// — TIPOS DE DATO —
T_INT : [Ii][Nn][Tt] | [Ii][Nn][Tt][Ee][Gg][Ee][Rr] ;
T_VARCHAR : [Vv][Aa][Rr][Cc][Hh][Aa][Rr] ;

// — NÚMEROS —
NUMERO_DECIMAL : [0-9]+ '.' [0-9]+ ;
NUMERO_ENTERO : [0-9]+ ;

// — CADENA DE TEXTO —
CADENA_TEXTO : '\'' ( '\\' | ~['\\'] )* '\'' ;

// — IDENTIFICADOR —
IDENTIFICADOR : [a-zA-Z_] [a-zA-Z0-9_]* ;
```

```
// — IGNORADOS —
COMENTARIO_LINEA : '--' ~[\r\n]* -> skip ;
COMENTARIO_BLOQUE : '/*' .*? '*/' -> skip ;
ESPACIO : [ \t\r]+ -> skip ;
NUEVA_LINEA : '\n' ;

// — ERROR (red de seguridad) —
ERROR_LEXICO : . ;
```

4.2 Reglas léxicas — SQL.g4 (archivos generados por ANTLR)

Una vez escrito el SQL.g4, se ejecuta el siguiente comando para generar automáticamente todos los archivos del lexer:

```
java -jar antlr-4.11.1-complete.jar -Dlanguage=Python3 -visitor -listener
SQL.g4
```

Este único comando genera automáticamente: SQLLexer.py (el motor del lexer), SQLParser.py (el parser), SQLListener.py (interfaz del patrón listener), SQLVisitor.py (interfaz del patrón visitor), SQL.interp, SQL.tokens, SQLLexer.interp y SQLLexer.tokens.

SQLLexer.py contiene la gramática compilada como una lista de números llamada ATN serializado (Augmented Transition Network), que es el autómatas de estados que el lexer ejecuta en tiempo real para reconocer tokens. También define las constantes numéricas de cada token: SELECT = 1, FROM = 2, y así hasta ERROR_LEXICO = 136. Nunca se edita manualmente.

4.3 Lógica del analizador léxico — SQLLexer.py

El motor del lexer es generado por ANTLR y hereda de la clase Lexer del runtime. Su funcionamiento interno se basa en el ATN (Augmented Transition Network): una estructura de autómatas que procesa los caracteres de entrada uno a uno y decide qué token producir en cada momento. El LexerATNSimulator es el componente que ejecuta ese autómata.

El lexer aplica siempre la primera regla que coincida con el texto actual. Por eso el orden en la gramática es crítico: NUMERO_DECIMAL debe aparecer antes que NUMERO_ENTERO para que 3.14 no se tokenice como tres piezas separadas. Las palabras reservadas deben aparecer antes que IDENTIFICADOR para que SELECT no sea reconocido como nombre de variable. ERROR_LEXICO va al último como red de seguridad.

4.4 Manejo y reporte de errores — main.py

El archivo main.py es el único que el equipo escribió completamente. Conecta todos los componentes generados por ANTLR y presenta los resultados en formato legible.

```
import sys
import os
from antlr4 import CommonTokenStream, InputStream
from antlr4.error.ErrorListener import ErrorListener
from SQLLexer import SQLLexer

class LexerErrorListener(ErrorListener):
    def __init__(self):
        super().__init__()
        self.errores = []

    def syntaxError(self, recognizer, offendingSymbol, line, column, msg, e):
        self.errores.append({
            'linea': line,
            'columna': column + 1,
            'mensaje': msg
        })

NOMBRES = {
    # DML
    SQLLexer.SELECT: 'SELECT',
    SQLLexer.FROM: 'FROM',
    SQLLexer.WHERE: 'WHERE',
    SQLLexer.INSERT: 'INSERT',
    SQLLexer.INTO: 'INTO',
    SQLLexer.VALUES: 'VALUES',
    SQLLexer.UPDATE: 'UPDATE',
    SQLLexer.SET: 'SET',
    SQLLexer.DELETE: 'DELETE',
    SQLLexer.MERGE: 'MERGE',
    SQLLexer.TO: 'TO',
    etc.... para todos los tipos
```

```

# Tokens que se omiten en la salida
OMITIR = {'COMENTARIO_LINEA', 'COMENTARIO_BLOQUE', 'ESPACIO', 'NUEVA_LINEA', 'EOF'}

#análisis

def analizar(fuente: str, nombre_archivo: str = "<fuente>"):
    print("=" * 50)
    print(f"  Analizador Lexico SQL")
    print(f"  Archivo: {nombre_archivo}")
    print("=" * 50)

    input_stream = InputStream(fuente)
    lexer = SQLLexer(input_stream)

    error_listener = LexerErrorListener()
    lexer.removeErrorListeners()
    lexer.addErrorListener(error_listener)

    stream = CommonTokenStream(lexer)
    stream.fill()
    tokens = stream.tokens

    print("\nToken List:")

    contador = 1
    for tok in tokens:
        nombre = NOMBRES.get(tok.type, f'TIPO_{tok.type}')
        if nombre in OMITIR:
            continue

        valor = f'"{tok.text}"'

        print(f"  {contador}: ({nombre}, {valor})")
        contador += 1

    print(f"\nTotal de tokens: {contador - 1}")

    # Recolectar tokens ERROR_LEXICO
    for tok in tokens:
        if NOMBRES.get(tok.type, '') == 'ERROR_LEXICO':
            error_listenererrores.append({
                'linea': tok.line,
                'columna': tok.column + 1,
                'mensaje': f"Símbolo desconocido '{tok.text}'"
            })

    if error_listenererrores:
        print("\nError List:")
        for i, err in enumerate(error_listenererrores, 1):
            print(
                f"  Line {err['linea']}, Column {err['columna']}: "
                f"{err['mensaje']}"
            )
        print(f"\nTotal de errores: {len(error_listenererrores)}")
    else:
        print("\nAnálisis lexico completado sin errores.")

    print()
    return tokens, error_listenererrores

```

Sección 5 — Demostración Funcional

Las siguientes capturas muestran la ejecución del lexer desde PowerShell en Windows. Cada escenario fue ejecutado con python main.py desde la carpeta del proyecto (Python 3.12, antlr4-python3-runtime 4.11.1).

5.1 Ejecución con consulta válida

Se tokeniza el archivo consulta_valida.sql, que contiene un SELECT con CTE (WITH), funciones de ventana (OVER, PARTITION BY), CASE/WHEN/ELSE, GROUP BY y subconsulta. El archivo no contiene errores léxicos.

Comando ejecutado:

```
PS C:\WINDOWS\system32> cd C:\Users\memom\Documents\SQLexer
PS C:\Users\memom\Documents\SQLexer> python main.py consulta_valida.sql
=====
Analizador Lexico SQL
Archivo: consulta_valida.sql
=====

Token List:
1: (WITH, "WITH")
2: (IDENTIFICADOR, "EstadisticasDept")
3: (AS, "AS")
4: (PAREN_IZQ, "(")
5: (SELECT, "SELECT")
6: (IDENTIFICADOR, "e")
7: (PUNTO, ".")
8: (IDENTIFICADOR, "dept_id")
9: (COMA, ",")
10: (COUNT, "COUNT")
11: (PAREN_IZQ, "(")
12: (IDENTIFICADOR, "e")
13: (PUNTO, ".")
14: (IDENTIFICADOR, "id")
15: (PAREN_DER, ")")
16: (AS, "AS")
17: (IDENTIFICADOR, "total_empleados")
18: (COMA, ",")
19: (AVG, "AVG")
20: (PAREN_IZQ, "(")
21: (IDENTIFICADOR, "e")
22: (PUNTO, ".")
23: (IDENTIFICADOR, "salario")
24: (PAREN_DER, ")")
25: (OVER, "OVER")
26: (PAREN_IZQ, "(")
27: (PARTITION, "PARTITION")
28: (BY, "BY")
29: (IDENTIFICADOR, "e")
```

```
30: (PUNTO, ".")
31: (IDENTIFICADOR, "dept_id")
32: (PAREN_DER, ")")
33: (AS, "AS")
34: (IDENTIFICADOR, "salario_promedio")
35: (COMA, ",")
36: (CASE, "CASE")
37: (WHEN, "WHEN")
38: (IDENTIFICADOR, "e")
39: (PUNTO, ".")
40: (IDENTIFICADOR, "salario")
41: (MAYOR, ">")
42: (NUMERO_ENTERO, "100000")
43: (THEN, "THEN")
44: (CADENA_TEXTO, "'Alto'")
45: (ELSE, "ELSE")
46: (CADENA_TEXTO, "'Normal'")
47: (END, "END")
48: (AS, "AS")
49: (IDENTIFICADOR, "clasificacion")
50: (FROM, "FROM")
51: (IDENTIFICADOR, "empleados")
52: (AS, "AS")
53: (IDENTIFICADOR, "e")
54: (WHERE, "WHERE")
55: (IDENTIFICADOR, "e")
56: (PUNTO, ".")
57: (IDENTIFICADOR, "activo")
58: (IGUAL, "=")
59: (IDENTIFICADOR, "TRUE")
60: (GROUP, "GROUP")
61: (BY, "BY")
62: (IDENTIFICADOR, "e")
63: (PUNTO, ".")
64: (IDENTIFICADOR, "dept_id")
65: (COMA, ",")
66: (IDENTIFICADOR, "e")
67: (PUNTO, ".")
68: (IDENTIFICADOR, "salario")
69: (PAREN_DER, ")")
```

Total de tokens: 69

Análisis léxico completado sin errores.

El lexer produjo 69 tokens clasificados correctamente.

5.2 Ejecución con errores léxicos

Se tokeniza el archivo consulta_errores.sql, que contiene símbolos no permitidos: @, \$, #, ~ y ?.

python main.py consulta_errores.sql

```
PS C:\Users\memom\Documents\SQLexer> python main.py consulta_errores.sql
=====
Analizador Lexico SQL
Archivo: consulta_errores.sql
=====

Token List:
1: (DECLARE, "DECLARE")
2: (ERROR_LEXICO, "@")
3: (IDENTIFICADOR, "mi_variable")
4: (T_INT, "INT")
5: (PUNTO_COMA, ";")
6: (SELECT, "SELECT")
7: (IDENTIFICADOR, "id")
8: (COMA, ",")
9: (IDENTIFICADOR, "nombre")
10: (COMA, ",")
11: (IDENTIFICADOR, "salario")
12: (ERROR_LEXICO, "$")
13: (FROM, "FROM")
14: (IDENTIFICADOR, "empleados")
15: (ERROR_LEXICO, "#")
16: (IDENTIFICADOR, "temporales")
17: (WHERE, "WHERE")
18: (IDENTIFICADOR, "activo")
19: (IGUAL, "=")
20: (IDENTIFICADOR, "TRUE")
21: (PUNTO_COMA, ";")
22: (WITH, "WITH")
23: (IDENTIFICADOR, "Ventas")
24: (AS, "AS")
25: (PAREN_IZQ, "(")
26: (SELECT, "SELECT")
27: (IDENTIFICADOR, "vendedor_id")
28: (COMA, ",")
29: (IDENTIFICADOR, "total")
30: (ERROR_LEXICO, "~")
31: (IDENTIFICADOR, "descuento")
32: (AS, "AS")
33: (IDENTIFICADOR, "total_real")
34: (FROM, "FROM")
35: (IDENTIFICADOR, "ventas_diarias")
36: (PAREN_DER, ")")
37: (SELECT, "SELECT")
38: (MULT, "*")
39: (FROM, "FROM")
40: (IDENTIFICADOR, "Ventas")
41: (PUNTO_COMA, ";")
```

```
42: (GRANT, "GRANT")
43: (SELECT, "SELECT")
44: (ON, "ON")
45: (ERROR_LEXICO, "?")
46: (IDENTIFICADOR, "tabla_desconocida")
47: (TO, "TO")
48: (IDENTIFICADOR, "usuario")
49: (PUNTO_COMA, ";")

Total de tokens: 49

Error List:
Line 1, Column 9: Símbolo desconocido '@'
Line 3, Column 27: Símbolo desconocido '$'
Line 4, Column 15: Símbolo desconocido '#'
Line 10, Column 15: Símbolo desconocido '~'
Line 15, Column 17: Símbolo desconocido '?'

Total de errores: 5
```

El lexer detectó y reportó los 5 errores léxicos con línea y columna exacta:

- Línea 2, columna 9: Símbolo desconocido '@'
- Línea 5, columna 27: Símbolo desconocido '\$'
- Línea 6, columna 15: Símbolo desconocido '#'
- Línea 13, columna 15: Símbolo desconocido '~'
- Línea 19, columna 17: Símbolo desconocido '?'

El lexer no se detuvo al encontrar el primer error: continuó analizando el resto del archivo y reportó todos los errores al final. Los tokens válidos que rodeaban los símbolos inválidos fueron reconocidos correctamente.

Sección 6 — Conclusiones

Macías Tijerina Guillermo Antonio. El aprendizaje central del proyecto fue entender que la gramática SQL.g4 funciona como el "diccionario" que le damos a ANTLR para que construya el motor del lexer automáticamente. Definir las reglas en el archivo .g4 y ver cómo ANTLR genera el lexer, el parser, el listener y el visitor a partir de ellas fue lo que me permitió entender en la práctica cómo funciona la generación automática de analizadores.

Pecero Acevedo Héctor Humberto. El reto más interesante fue comprender el orden de las reglas en la gramática. Aprendí que ANTLR aplica siempre la primera regla que coincida, lo que hace que el orden sea crítico: NUMERO_DECIMAL debe ir antes que NUMERO_ENTERO, y las palabras reservadas antes que IDENTIFICADOR. Si el orden está mal, SELECT se reconoce como identificador y 3.14 se tokeniza como tres piezas separadas. Verlo fallar y corregirlo cambiando solo el orden en el .g4 fue la forma más efectiva de entender ese concepto.

Ponce Medina Ángel. Me llamó la atención el mecanismo de ERROR_LEXICO en la gramática. La regla ERROR_LEXICO : . al final del archivo .g4 acepta cualquier carácter que ninguna otra regla reconoció. Va al último precisamente porque ANTLR aplica la primera coincidencia. Esta red de seguridad permite que el lexer no se detenga al encontrar un símbolo inválido, sino que lo capture, lo reporte y continúe analizando el resto del archivo.

Jiménez Blanco José Antonio. Lo que aprendí al trabajar en el main.py fue la importancia de los diccionarios NOMBRES y CATEGORIAS. ANTLR internamente identifica los tokens por números enteros, no por nombres. Sin el diccionario NOMBRES que traduce esos números a texto legible, y sin CATEGORIAS que los agrupa semánticamente en DML, DDL, TCL, FUNCION, etc., la salida sería incomprensible. Construir esa capa de presentación fue lo que hizo el proyecto útil y entendible para el usuario final.

Sección 7 — Referencias

[1] Parr, T. (2013). The Definitive ANTLR 4 Reference. Pragmatic Bookshelf.

[2] Python Software Foundation. (2024). antlr4-python3-runtime.

Recuperado de <https://pypi.org/project/antlr4-python3-runtime/>

[3] ANTLR Project. (2024). ANTLR4 Documentation. Recuperado de

<https://github.com/antlr/antlr4/blob/master/doc/index.md>

[4] Python Software Foundation. (2024). warnings — Warning control.

Recuperado de <https://docs.python.org/3/library/warnings.html>